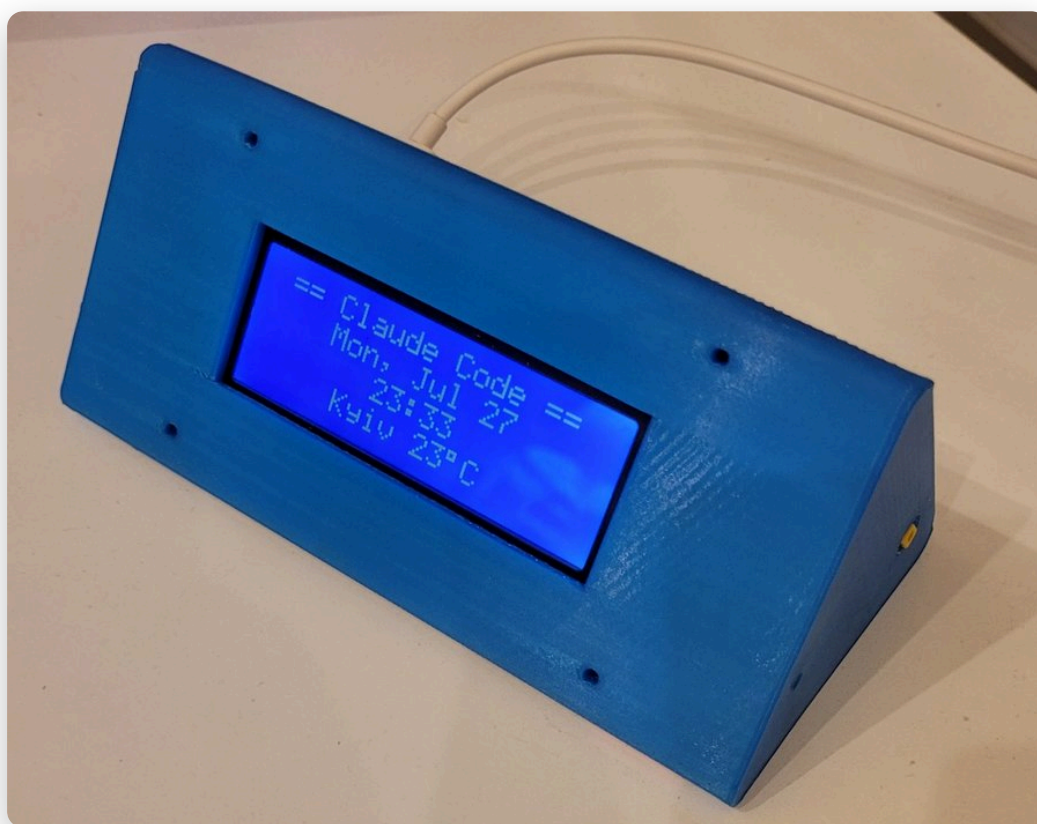


CLAUDIUS I

Hardware status indicator for Claude Code
User Guide



Plainstack technologies · Chaos into systems.
2026

1. What Claudius I is

Claudius I is a desktop device that shows, in real time, what Claude Code is doing on your computer. One glance at the screen tells you whether Claude is free, working, has asked a question, or is waiting for a permission. The device also shows your plan-usage limit, a clock with local weather, and — for cities in Ukraine — air-raid alerts with an audible siren.

It works the same way regardless of how you use Claude Code — from a terminal, or from an IDE extension (VS Code, JetBrains, and others): both share the same hooks the device listens to.

The screen is a classic 20×4 character LCD (HD44780). Every screen mock-up in this guide is drawn with the very same 5×8-dot character generator the physical display itself uses.

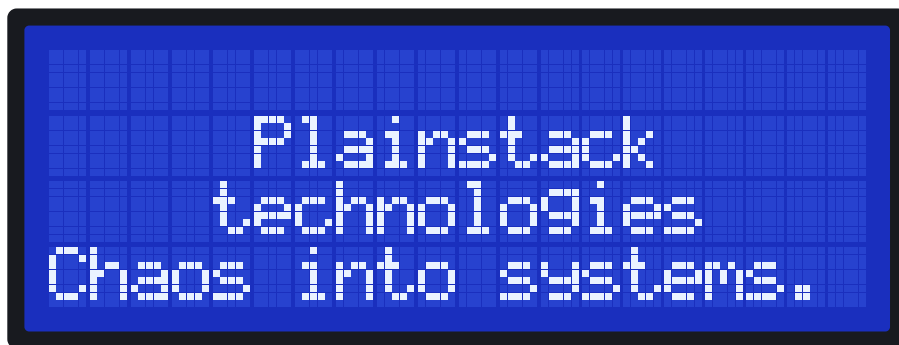
2. What's in the box, and requirements

- the Claudius I device;
- a USB-C cable that carries data (not power-only);
- a USB drive with the installer and this guide.

What your computer needs:

- Windows 10/11, Linux, or macOS 11+;
- Claude Code installed and logged in (<https://claude.com/claude-code> — run it at least once and sign in *before* setting up the device).

Plug the device into your computer with the USB-C cable — the screen lights up, shows a boot splash, then the clock. No separate power adapter is needed; any USB port works, the device is found automatically.



Boot splash screen

3. Installing the software

3.1. Windows — 5 minutes

1. Plug in the device with the USB-C cable and insert the USB drive.
2. Open the `ClaudeStatus` folder on the drive and double-click `ClaudeStatusSetup.exe`.
3. Click through the installer — no PowerShell command or admin rights needed for this part. It installs the app under `%LOCALAPPDATA%\Programs\ClaudeStatus`, adds a Start Menu entry, and opens the setup wizard in your browser as soon as it's done.

4. The wizard's first page just waits for the device — plug it in via USB-C now if you haven't already; the page finds it on its own, installs Python 3 if it's missing, replaces an incompatible USB driver if needed (this may show **one** administrator confirmation — accept it), and flashes the latest firmware, with nothing for you to click.
5. The wizard then asks you to pick your city on the map (for weather and alerts) and set brightness and sound preferences — brightness changes show live on the device itself as you drag the slider.
6. When the wizard shows *All set!*, the device shows the same message. Start using Claude Code — from a terminal (`cclaude`) or from the Claude Code extension in your IDE (VS Code, JetBrains, etc.); both work the same way — and the screen starts updating on the first response. (If Claude isn't started within a couple of minutes, the device falls back to the clock on its own — see 4.1.)

3.2. Linux / macOS — 5 minutes

1. Plug in the device. No driver is needed: Linux has one built into the kernel, macOS 11+ ships one with the OS.
2. Copy the `ClaudeStatus` folder from the drive to your computer, open a terminal in it, and run: `bash setup.sh`
3. The script installs python3, pyserial and avrdude through your package manager (it may ask for your sudo password), flashes the device, and asks the same questions in the terminal: city, brightness, sounds.
4. On Linux you'll be added to the `dialout` group — that becomes permanent after you log out and back in, but the installer also grants temporary access right away, so everything works without a re-login.
5. Start using Claude Code — `cclaude` in a terminal, or the Claude Code extension in your IDE (VS Code, JetBrains, etc.); either way the device picks it up automatically.

3.3. Changing settings, and uninstalling

- To change city, brightness, or sounds — just run the installer again (on Windows, the *Claudius I* Start Menu entry, or `ClaudeStatusSetup.exe` again); the device will remember the new settings.
- Brightness can also be changed without the wizard: `python b1.py 50` (0–100) from the app folder.
- Full removal: on Windows, use *Settings* → *Apps* → *Claudius I* (or *Add or Remove Programs*) — it cleans up the daemon, scheduled task, and hooks automatically. On Linux/macOS, run `bash uninstall.sh` from the `ClaudeStatus` folder.

3.4. Installer options (optional)

The table below lists flags for the underlying `setup.ps1` / `setup.sh` scripts — useful if something wasn't detected automatically, or if you want to skip the interactive wizard. On Windows, run `setup.ps1` directly instead of the .exe to use them (found in `%LOCALAPPDATA%\Programs\ClaudeStatus` after installing, or in the `ClaudeStatus` folder on the drive):

Windows (<code>setup.ps1</code>)	Linux / macOS (<code>setup.sh</code>)	Effect
<code>-Port COM7</code>	<code>--port /dev/ttyUSB0</code>	set the device port manually instead of auto-detecting
<code>-City "Berlin"</code>	— (city is asked in the terminal)	set the city without the interactive wizard

<code>-NoWeather</code>	<code>--no-weather</code>	disable weather and air-raid alerts entirely
<code>-NoAutostart</code>	<code>--no-autostart</code>	don't add a background auto-start entry at login

Example: `.\setup.ps1 -City "Kyiv" -NoAutostart` or `bash setup.sh --no-weather`.

3.5. What gets installed, and where

Component	Windows	Linux / macOS
App files	<code>%LOCALAPPDATA%\ClaudeStatus\app</code>	<code>~/.local/share/claude-status/app</code>
Claude Code hooks	<code>~/.claude/settings.json</code> (an existing file is backed up first)	
Autostart	Task Scheduler entry «ClaudeStatusDaemon»	systemd user service (Linux) / LaunchAgent (macOS)
Log file	<code>%TEMP%\claude_lcd_daemon.log</code>	<code>/tmp/claude_lcd_daemon.log</code>
Python 3, pyserial, avrdude	installed automatically if missing	

For the curious: technical details and architecture live in `readme.txt` and `ClaudeStatus/README.md` on the drive. For installation and everyday use, this guide is all you need.

4. Screens and states

4.1. All set! (setup finished)

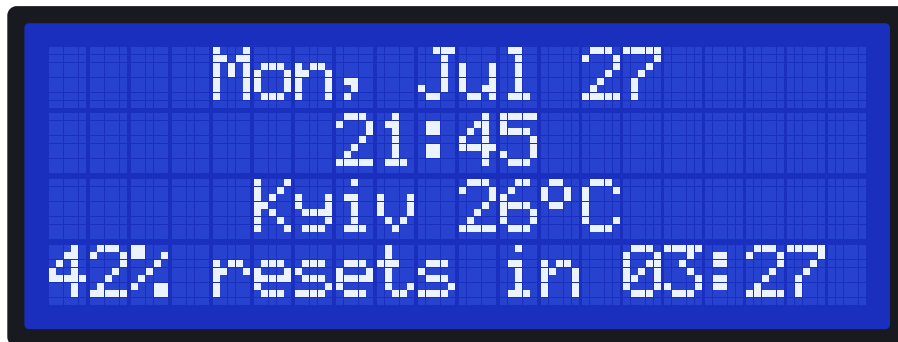
Shown the moment the browser setup wizard finishes — before you've even closed the browser tab — with a short ascending chime. It's a cue to start using Claude Code — from a terminal or your IDE extension. If Claude hasn't been started within a couple of minutes, the device gives up waiting and falls back to the clock (4.2) on its own.



Setup just finished

4.2. Clock (idle screensaver)

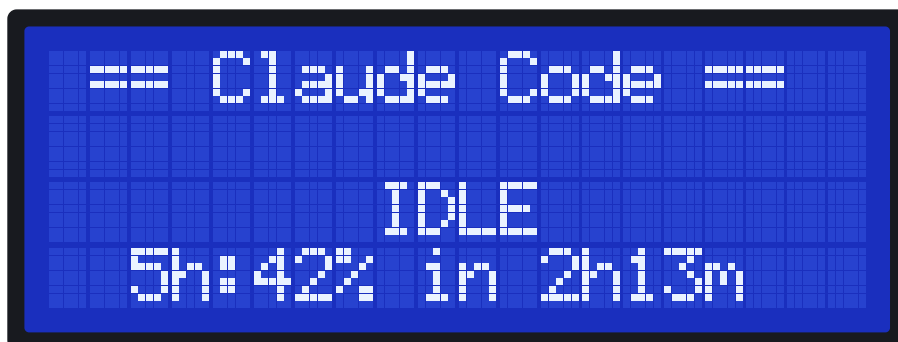
Shown while Claude Code isn't running, and also after 5 minutes of inactivity: date, time, city and temperature (refreshed once a minute), and your 5-hour usage percentage with the time until it resets on the bottom row. Any activity from Claude immediately restores the working screen.



Clock with date and weather

4.3. IDLE — Claude is free

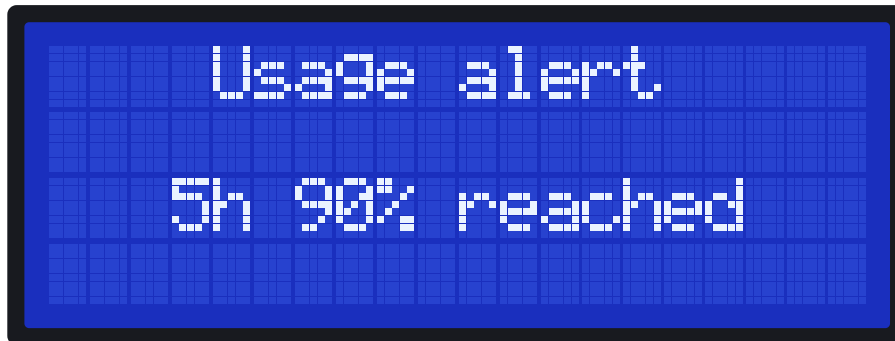
A Claude Code session is open with no task running. The bottom row shows your 5-hour plan-usage percentage and the time until it resets (the same numbers shown on [claude.ai](#) → Usage) — briefly "awaiting limit info" right after startup, until the first usage fetch lands.



Idle state with the usage line

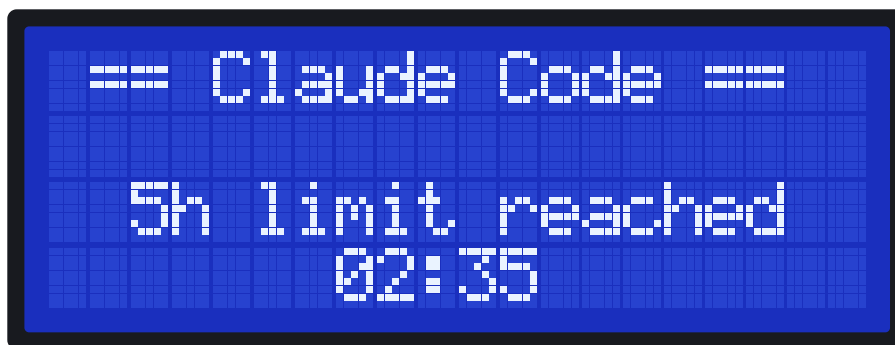
4.4. Approaching and reaching a usage limit

As your 5-hour or weekly usage crosses 75% and 90%, the device briefly shows what changed alongside the warning beep — so a beep heard during the screensaver is never a mystery:



Brief heads-up shown with the 75%/90% beep

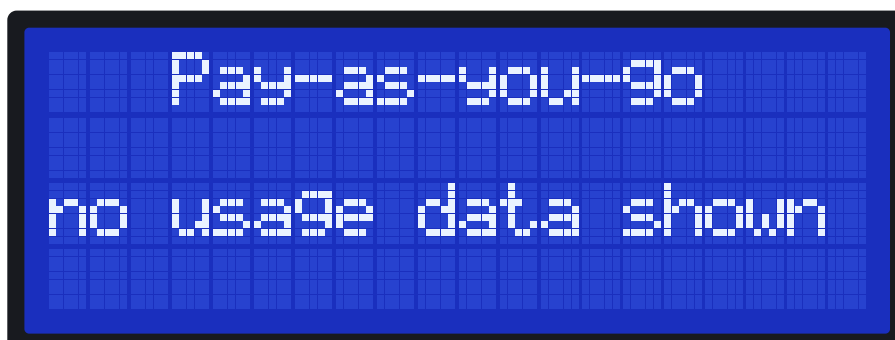
Once a window is completely used up, the screen switches to a dedicated limit screen with a live countdown to when it resets — HH:MM for the 5-hour limit, Xd:HH:MM for the weekly one:



5-hour limit reached, resets in 2h35m

The moment the window resets, a chime plays and the device returns to IDLE on its own — no action needed.

On a pay-as-you-go plan (an API key instead of a Claude subscription) there's no 5-hour/weekly limit to track. The device shows a one-time notice instead of the usage line, and otherwise behaves exactly the same — WORKING, IDLE, the clock screensaver, WAITING FOR INPUT and PERMISSION? all work normally.



One-time notice on a pay-as-you-go plan

4.5. WORKING — Claude is working

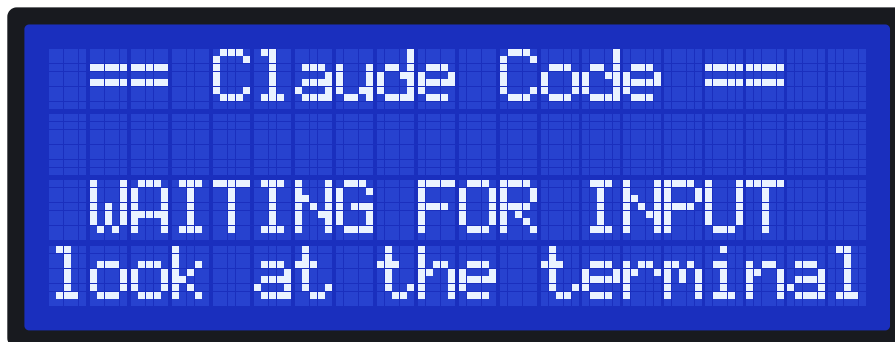
The top row shows the name of the project (folder) Claude is working in. The center shows a "verb" describing the work, matching Claude Code's own animated label (a different one each time). The bottom row is the 5-hour usage bar.



Working on project "my-project", 42% of the limit used

4.6. WAITING FOR INPUT — Claude is waiting for a reply

Claude has asked a question — in the terminal or your IDE extension. The device plays a short rising chirp; a message longer than 20 characters scrolls as a marquee, and the backlight switches to full brightness for the duration to draw your attention.



Waiting for the user's reply

4.7. PERMISSION? — a permission request

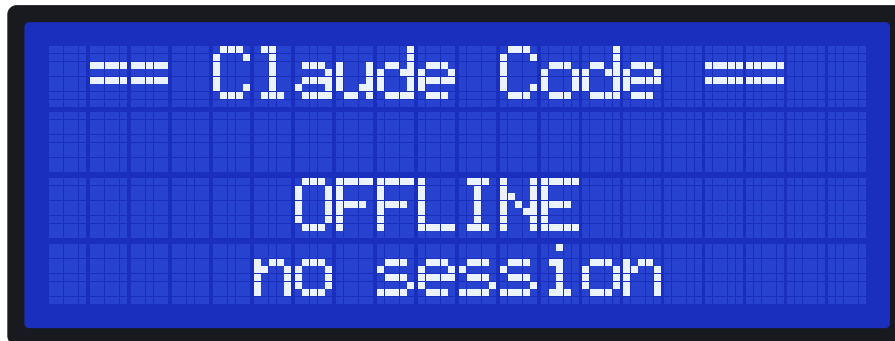
Claude is asking permission for an action (running a command, editing a file, etc). A distinct sound plays. As soon as you answer — in the terminal or your IDE — and Claude continues, the screen returns to WORKING on its own.



Requesting permission to run a command

4.8. OFFLINE — no session

No running Claude Code session was found. Start one — from a terminal (`claude`) or your IDE extension — and the state changes automatically.



Claude Code isn't running

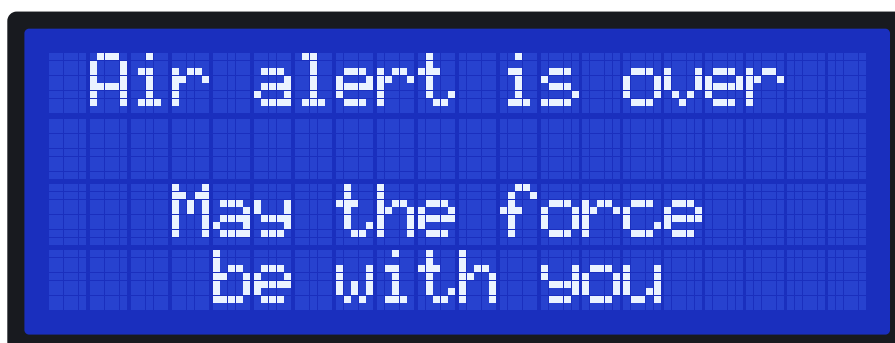
4.9. Air-raid alert (for cities in Ukraine)

If a city in Ukraine is selected in the settings, the device watches for air-raid alerts in your oblast through open public feeds (no accounts or keys needed). A siren plays and a warning screen appears the moment an alert starts.



Air-raid alert screen

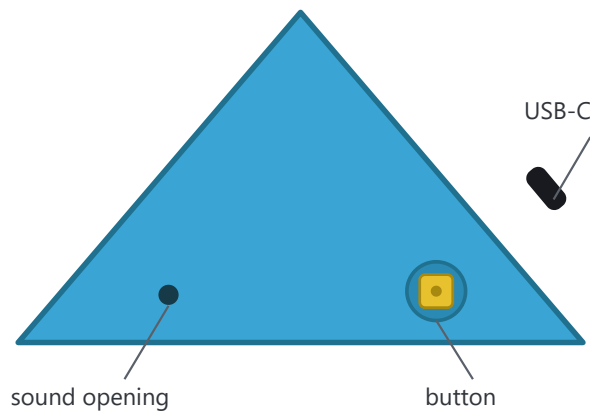
Pressing the button dismisses the alert screen (the sound won't repeat), but while the alert is still active, the top row keeps flashing `AIR RAID ALERT` as a reminder. A short melody plays once the all-clear comes:



Air-raid all-clear

5. The button

There is a button on the right side face of the enclosure; to its left is a sound opening (don't cover it). The USB-C connector sits near the back edge.



The device's right side face (schematic)

- **Short press** — turn the display off or on. When the display is off, all sounds are muted too.
- **During an alert** — a short press dismisses the alert screen.
- **If the screen went dark** (e.g. after the computer went to sleep) — a short press brings it back.
- **Long press (hold for 1 second)** — peek at your 7-day usage: the current screen is briefly replaced with the weekly percentage and time until it resets, for 5 seconds, then returns to whatever was showing. Works from any screen. Holding past 1 second fires the peek immediately (no need to release first) — releasing afterwards doesn't also trigger the short-press action above.

6. Sound signals

Event	Sound
The setup wizard finishes (<i>All set!</i>)	ascending run (C6 → D6 → E6 → G6 → C7)
Claude finished working	two short taps (C7, C7)
Claude is waiting for a reply	rising fifth (D → A)
Permission request	a distinct short pattern
75% of the limit used (5-hour / 7-day)	double warning chirp, with a brief on-screen message
90% of the limit used	triple cascade, with a brief on-screen message
A usage window resets after being fully used	major arpeggio, then back to IDLE
Air-raid alert starts	siren
Alert all-clear	major arpeggio

Sounds can be turned off in the setup wizard (re-run the installer), or temporarily by turning off the display with the button.

7. Display automation

- The screen blanks together with your computer's monitors and wakes up with them — on Windows, macOS, and Linux (X11 and GNOME are supported on Linux).
- After 5 minutes in IDLE, the clock screensaver turns on.

- The backlight temporarily switches to full brightness during a marquee-scrolling message (see section 4.6).
- Time and date are synced from your computer automatically on every connection.

8. If something isn't working

Symptom	What to do
Screen is dark	Check that the USB-C cable carries data (not charge-only); try another port; press the button.
States aren't updating	Make sure Claude Code is running (<code>cldude</code> in a terminal, or an active session in your IDE extension). Check the log at <code>%TEMP%\cldude_lcd_daemon.log</code> (Windows) or <code>/tmp/cldude_lcd_daemon.log</code> .
"Device not functioning" during setup (Windows)	Re-run the installer (<code>CldudeStatusSetup.exe</code> , or <code>setup.ps1</code> directly) — it replaces the incompatible CH340 driver with the working v3.4 from the drive.
Port "busy" or won't open	Unplug and re-plug the USB-C cable — the device is found automatically.

9. Technical specifications

Controller	Arduino Nano (ATmega328P)
Display	20×4 character LCD (HD44780) over I ² C, 5×8-dot character generator
Button	mechanical, on the right side face of the enclosure
Sound	piezo buzzer
Clock	synced from the computer automatically
Connection	USB-C (power + data)
Compatibility	Windows 10/11, Linux, macOS 11+